

Projeto EDA2:

Agrupamento de Notícias

Arquitetura Integrada, Decisões Técnicas e Implementação

Otimizada

Integrantes do Grupo

A divisão estratégica das responsabilidades do projeto



Giovani De Oliveira Teodoro Coelho: Arquitetura do Grafo Base



Artur Fernandes Galdino: Processamento de
Linguagem Natural e Matemática



João Eduardo de Souza Leles: Algoritmo de
Kruskal (O Coração do Agrupamento)



Davi Ursulino de Oliveira: Estruturas de Dados
Auxiliares (Union-Find e Hash)



Fábio Alessandro Santos Vieira: Dados, Testes e Apresentação

A Estrutura do Projeto

Divisão do nosso ambiente de desenvolvimento



src/

O núcleo do nosso projeto. Aqui ficam as pastas de estruturas (Grafo, Union-Find, Hash), processamento (PLN) e algoritmos (Kruskal).



data/

Base de testes centralizada. Armazena o noticias.json, nossa massa de dados gerada sinteticamente para validação.



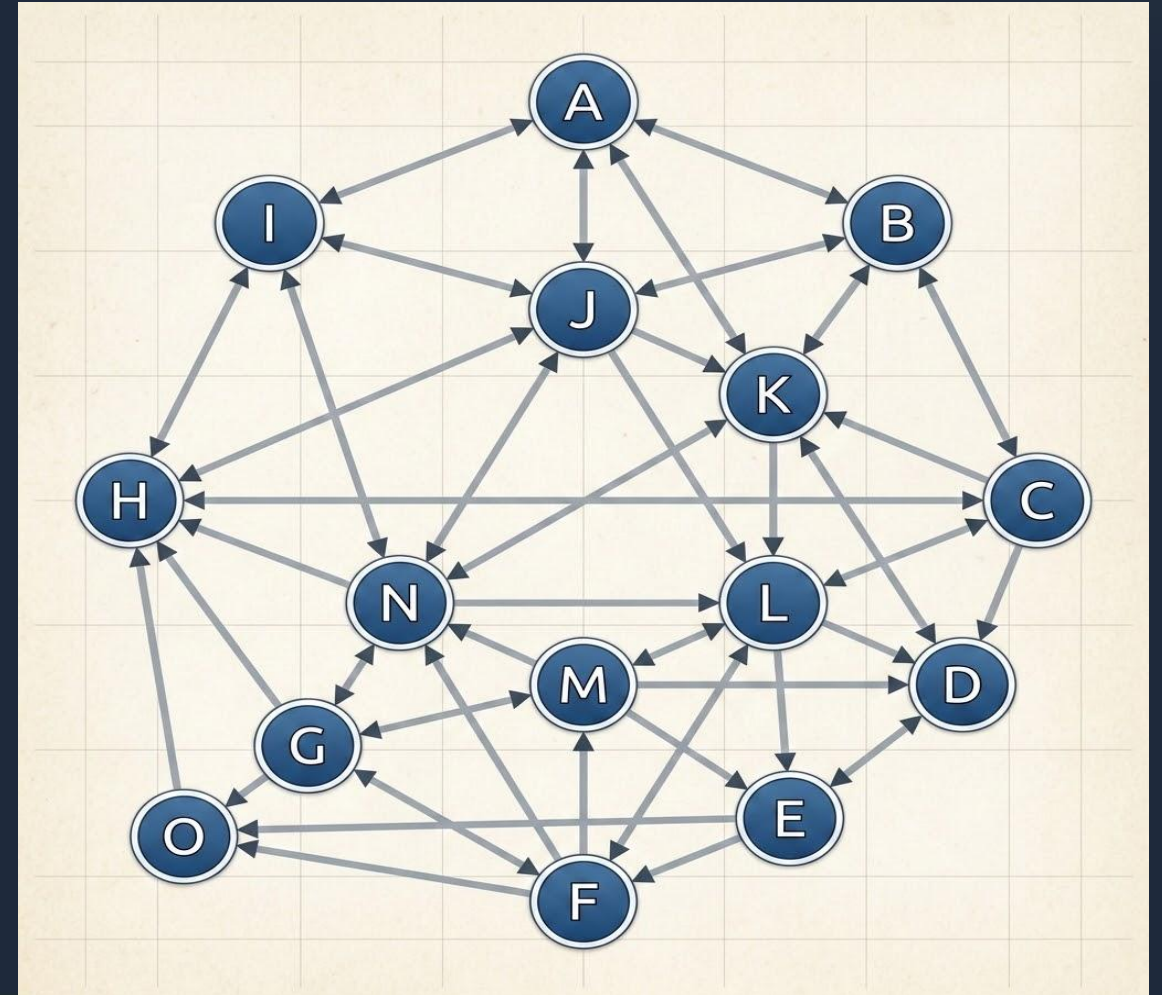
test/

Scripts de análise e validação que cruzam a saída do Algoritmo de Kruskal e avaliam a qualidade dos agrupamentos formados.

Grafo Base

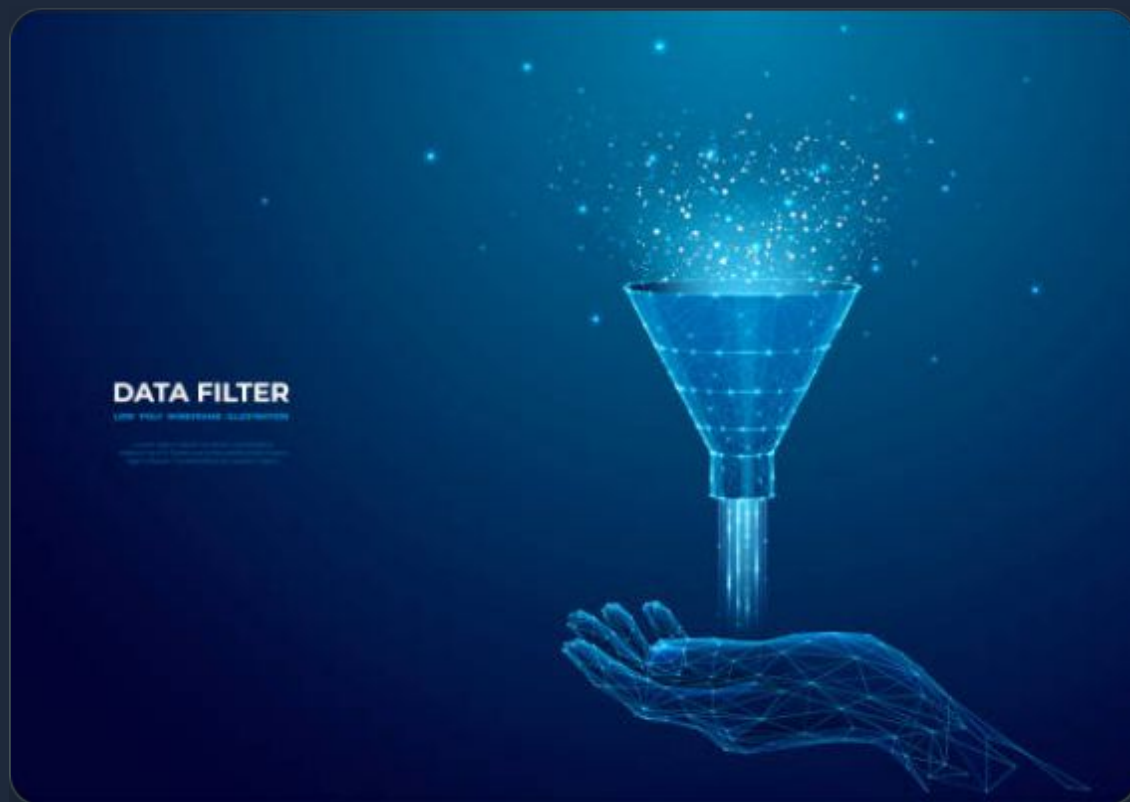
Implementação manual com Listas de Adjacência

- **Construção Zero-Libs:** Criação da classe Grafo utilizando um dicionário nativo do Python, onde a chave é o ID e o valor é uma lista de conexões.
- **Recepção de Dados:** O grafo recebe as distâncias processadas externamente e monta a rede que interliga as notícias.
- **Desafio:** Evitar que o grafo cresça descontroladamente, consumindo toda a memória do sistema.



O Benefício do Ponto de Corte

Como evitamos a explosão de memória

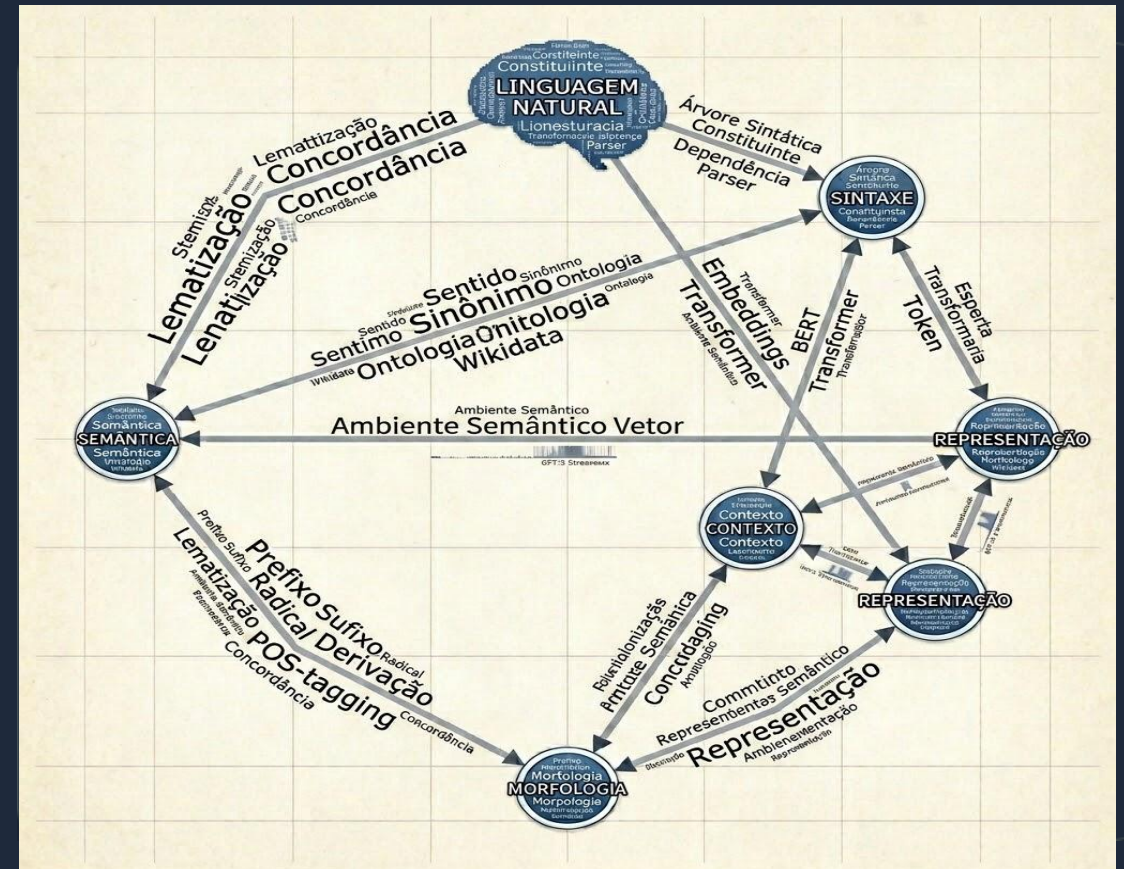


- **O Filtro de Densidade:** A função adicionar_aresta possui uma trava embutida. Conexões com similaridade menor que 0.20 são bloqueadas.
- **Grafo Esparso vs Denso:** Sem o corte, teríamos um grafo completo (Denso), travando o computador. O filtro mantém a estrutura Esparsa.
- **O Resultado Teórico:** Garante que as operações futuras do sistema continuem rodando em um tempo ótimo de $O(V+E)$.

PLN e Matemática

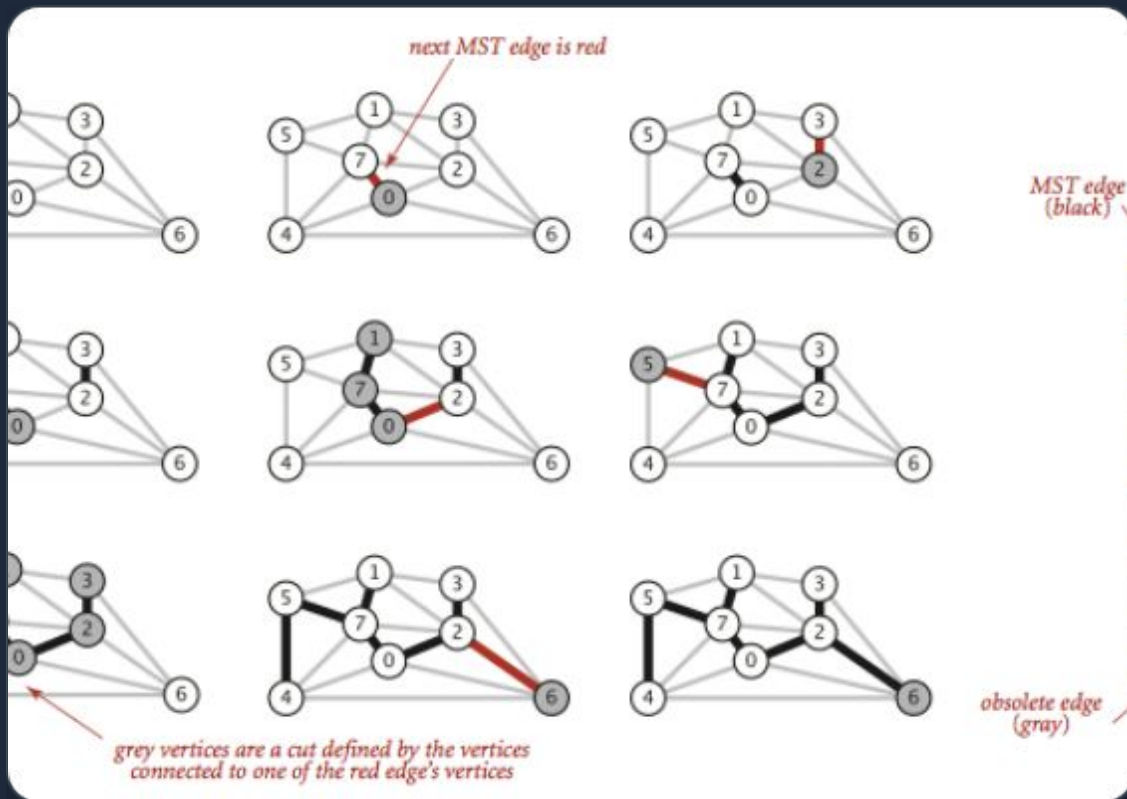
Preparação dos dados e cálculos de arestas

- **Limpeza Textual (PLN):** Remoção de pontuações e "stop words" (como "e", "ou", "de") para focar exclusivamente na semântica das notícias.
- **Cálculo de Similaridade:** Comparação das notícias duas a duas para obter uma métrica precisa do quão parecidas elas são.
- **Matemática para o Kruskal:** Converteremos a Similaridade em Distância através da fórmula **Distância = 1 - Similaridade**, pois o Kruskal busca o *menor caminho*.



Algoritmo de Kruskal

O coração do nosso agrupamento



- **Extração e Ordenação:** Pega todas as arestas passadas pela Pessoa 1 e ordena da menor para a maior distância em tempo $O(E \log V)$.
- **Construção Segura:** Percorre a lista ordenada acionando a estrutura Union-Find para ligar os nós e construir a Árvore Geradora Mínima (MST) sem loops.
- **O Objetivo:** Transformar um emaranhado de conexões em um caminho único e minimizado que alcance todas as notícias.

Superando o Efeito Encadeamento

A estratégia inteligente do Kruskal para formar editorias.



O Problema

O *Efeito Encadeamento* acontece quando uma "notícia ponte" (ex: escândalo político no esporte) funde acidentalmente dois grandes temas distintos.



A Solução

Ao finalizar a MST, nós apagamos as *4 arestas mais pesadas* (as maiores distâncias) da árvore, eliminando conexões forçadas e fracas.



O Resultado

O corte nessas arestas destrói os nós pontes e quebra o grafo exatamente em 5 clusters isolados, dividindo as notícias em editorias perfeitas.

Estruturas Auxiliares

A base de alta velocidade que impede travamentos

Union-Find

Criado totalmente do zero para gerenciar conjuntos disjuntos.

- Implementação das funções essenciais Find e Union.
- Permite à Pessoa 3 saber instantaneamente se duas notícias criarão um loop fechado.
- Evita a necessidade de percorrer o grafo todo a cada nova aresta.

Tabela Hash

O tradutor entre títulos legíveis e a matemática do grafo.

- A classe Grafo lida apenas com **IDs Numéricos**.
- Uso de dicionários nativos do Python para criar uma busca **$O(1)$** .
- Quando o usuário busca o título, o mapa devolve o ID numérico na hora.

A Otimização Extrema do Union-Find

Como passamos de um tempo linear para um tempo quase constante



Compressão de Caminho: No Find, encurtamos a árvore apontando todos os nós visitados diretamente para a raiz. Isso impede que a árvore fique profunda.



Union por Tamanho: Ao unir conjuntos, anexamos sempre a árvore menor sob a maior. Isso evita formações longas em formato de "linha reta".



O Benefício Real: Sem essas técnicas, buscar ciclos (DFS) custaria $O(V)$. Com elas, o tempo cai para $O(\alpha(N))$, tornando a execução quase instantânea mesmo com milhares de textos.

Dados e Validação

Provando que a teoria virou realidade

- **Massa de Teste:** Geração via IA de um JSON contendo entre 50 a 100 notícias divididas claramente em temas (esporte, economia, tech).
- **Análise Analítica:** Execução do pipeline completo para cruzar dados. A saída do algoritmo separou corretamente as editorias?
- **Identificação de Falhas:** Mapear notícias que se perderam nos cortes para calibrar e justificar as decisões do time ao vivo.
- Montagem do discurso técnico para defesa arquitetural da solução.

```
[1/4] Processando textos e calculando distâncias de Jaccard...
      Total de combinações analisadas: 1770
[2/4] Montando o Grafo e aplicando Filtro de Densidade (Corte de 0.80)...
      Arestas válidas no grafo após o filtro: 554
[3/4] Executando Kruskal (Union-Find) para formar 3 tópicos...

=====
🔍 RESULTADO DO AGRUPAMENTO (COMUNIDADES ENCONTRADAS)
=====

Tópico 1 (Contém 19 notícias):
IDs: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]

Tópico 2 (Contém 21 notícias):
IDs: [39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 59, 60]

Tópico 3 (Contém 20 notícias):
IDs: [20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 58]

=====

[4/4] Alimentando Tabela Hash para busca O(1)...

=====
🔍 TESTE DE BUSCA O(1) NA TABELA HASH
=====

Sucesso! O título 'Mercado financeiro cai com decisão do banco sobre juros'
corresponde ao ID 20.
```

Dúvidas?

Fiquem à vontade para questionar nosso código, lógica e decisões matemáticas.

Projeto: meu_projeto_grafos | Matéria: Estrutura de Dados 2